

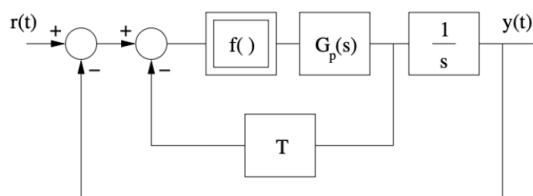
Sprawozdanie z projektu MMM

Stanisław Latuszek 203248, Jakub Wrzochol 197850

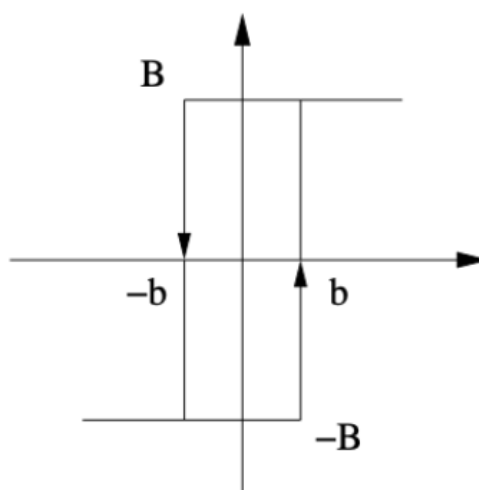
czerwiec 2026

1 Cel projektu

Celem projektu było opracowanie programu umożliwiającego symulację odpowiedzi układu nieliniowego na różne sygnały wejściowe. Program miał umożliwiać modyfikację parametrów układu oraz sygnałów wejściowych, a także wizualizację wyników symulacji.



Rys. 1 Schemat symulowanego układu gdzie $G_p = \frac{k+p}{1+T_p s}$



Rys. 2 Charakterystyka elementu nieliniowego $f()$ z symulowanego układu

2 Część teoretyczna

Układ nieliniowy, który symulowaliśmy, składa się z elementu liniowego oraz elementu nieliniowego opisanego funkcją $f()$, która posiada charakterystykę histerezową. Element liniowy jest typowym układem pierwszego rzędu, natomiast element nieliniowy wprowadza do układu nieliniowość, co powoduje, że odpowiedź układu na sygnały wejściowe jest bardziej złożona i może wykazywać różne zachowania w zależności od parametrów układu i sygnałów wejściowych.

Zaczęto od nazwania wszystkich sygnałów w układzie:

- $r(t)$ – sygnał wejściowy
- $e(t)$ – uchyb sterowania
- $e_1(t)$ – sygnał wejściowy histerezy
- $u(t)$ – sygnał wyjściowy histerezy
- $x_2(t)$ – sygnał wejściowy elementu liniowego
- $x_1(t)$ – sygnał wyjściowy układu

Następnie wyznaczono zależności między sygnałami w układzie.

$$\begin{aligned}
e(t) &= r(t) - x_1(t) \\
e_1(t) &= e(t) - T \cdot x_2(t) \\
u(t) &= \begin{cases} -B & \text{dla } e_1(t) < -b \\ B & \text{dla } e_1(t) > b \\ u(t-1) & \text{dla } -b \leq e_1(t) \leq b \end{cases} \\
X_2(s) &= \frac{k_p}{1 + T_p s} \cdot U(s) \\
X_1(s) &= \frac{1}{s} \cdot X_2(s)
\end{aligned}$$

Co po przekształceniach daje poniższy układ równań różniczkowych opisujący zachowanie układu nieliniowego:

$$\begin{aligned}
u(t) &= \begin{cases} -B & \text{dla } r(t) - x_1(t) - T x_2(t) < -b \\ B & \text{dla } r(t) - x_1(t) - T x_2(t) > b \\ u(t-1) & \text{dla } -b \leq r(t) - x_1(t) - T x_2(t) \leq b \end{cases} \\
\begin{cases} \dot{x}_1(t) = x_2(t) \\ \dot{x}_2(t) = -\frac{1}{T_p} \cdot x_2(t) + \frac{k_p}{T_p} \cdot u(t) \end{cases}
\end{aligned}$$

Przekształcenie problemu do postaci równań różniczkowych pozwoliło na zastosowanie szeregu metod numerycznych do symulacji działania układu. W celu osiągnięcia maksymalnej dokładności postanowiono zastosować metodę Rungego-Kutty czwartego rzędu z adaptacyjnym krokiem symulacji. Do tego celu wykorzystano metodę Dormanda-Prince'a, opisaną poniższą tabelą Butchera: Wykorzystując te współ-

0	0	0	0	0	0	0	0
$\frac{1}{5}$	$\frac{1}{5}$	0	0	0	0	0	0
$\frac{3}{10}$	$\frac{3}{40}$	$\frac{9}{40}$	0	0	0	0	0
$\frac{4}{5}$	$\frac{44}{45}$	$-\frac{56}{15}$	$\frac{32}{9}$	0	0	0	0
$\frac{8}{9}$	$\frac{19372}{6561}$	$-\frac{25360}{2187}$	$\frac{64448}{6561}$	$-\frac{212}{729}$	0	0	0
1	$\frac{9017}{3168}$	$-\frac{355}{33}$	$\frac{46732}{5247}$	$\frac{49}{176}$	$-\frac{5103}{18656}$	0	0
1	$\frac{35}{384}$	0	$\frac{500}{1113}$	$\frac{125}{192}$	$-\frac{2187}{6784}$	$\frac{11}{84}$	0
	$\frac{35}{384}$	0	$\frac{500}{1113}$	$\frac{125}{192}$	$-\frac{2187}{6784}$	$\frac{11}{84}$	0
	$\frac{5179}{57600}$	0	$\frac{7571}{16695}$	$\frac{393}{640}$	$-\frac{92097}{339200}$	$\frac{187}{2100}$	$\frac{1}{40}$

Tab. 1 Tabela Butchera dla metody Dormanda-Prince'a

czynniki obliczono kolejne parametry k_i , a później wyniki 5-tego i 4-tego rzędu. W ten sposób otrzymano po dwie estymacje dla obu zmiennych x_1 oraz x_2 . Obliczono błędy estymacji dla obu zmiennych poprzez porównanie wyników 5-tego i 4-tego rzędu, a następnie znormalizowane je względem tolerancji względnej i bezwzględnej wykorzystując wzór:

$$\text{err}_{\text{norm}} = \frac{\text{error}}{\text{tol}_{\text{abs}} + \text{tol}_{\text{rel}} \cdot \max(|x^{(5)}(t+h)|, |x(t)|)}$$

Ten sposób normalizacji unika problemów z dzieleniem przez zero w przypadku gdy $x(t)$ zbliża się do zera. Wybieramy także większą wartość pomiędzy estymacją a aktualną wartością, by zwiększyć stabilność w przypadku gwałtownych zmian w sygnale. Następnie na podstawie największego z dwóch znormalizowanych błędów dopuszczamy lub odrzucamy krok symulacji, a także dostosowujemy jego długość do aktualnej sytuacji w układzie. W celu osiągnięcia lepszej dokładności oraz zapobiegnięcia zatrzymania się symulacji dodano również maksymalną i minimalną długość kroku symulacji, a także maksymalną ilość iteracji.

Do wyznaczania następnego kroku symulacji wykorzystano wzór:

$$h_{\text{next}} = s \cdot h \cdot \left(\frac{1}{\text{err}_{\text{norm}}} \right)^{\frac{1}{5}}$$

gdzie s jest współczynnikiem bezpieczeństwa, a h poprzednią długością kroku.

W przypadku gdy krok jest akceptowany wykorzystywana jest estymacja 5-tego rzędu, uznawana za bardziej dokładną.

3 Implementacja programu

Aplikacja symulacyjna została zaimplementowana w języku Python z wykorzystaniem podejścia obiektowego oraz modułowego, co pozwoliło na ściśle odseparowanie warstwy obliczeniowej od logiki generowania sygnałów i interfejsu graficznego. Główną osią programu jest cykl symulacyjny uruchamiany w czasie rzeczywistym, w którym wejściowy sygnał wymuszający jest dynamicznie przetwarzany przez blok nieliniowy oraz liniowy obiekt dynamiczny, a wyniki są natychmiastowo wizualizowane na ekranie.

Wewnętrzna architektura systemu opiera się na klasie reprezentującej symulowany obiekt, która odpowiada za przechowywanie aktualnego stanu układu, historii sygnałów oraz parametrów strukturalnych. Numeryczne rozwiązanie nieliniowego układu równań różniczkowych realizowane jest za pomocą opisanego w poprzedniej sekcji algorytmie. Dobrano wartość tolerancji absolutnej równej $10V-7$, tolerancji względnej 10^{-5} , współczynnika bezpieczeństwa do adaptacyjnego kroku równego 0.9, oraz dopuszczalną długość kroku symulacji w zakresie od 10^{-8} a 0.05, dodatkowo ustalono maksymalną ilość iteracji na jedną próbkę sygnału wyjściowego na 10 milionów. Wartości te są łatwe w edycji i pozwalają na dostosowanie dokładności oraz wydajności symulacji do konkretnych potrzeb użytkownika z poziomu kodu.

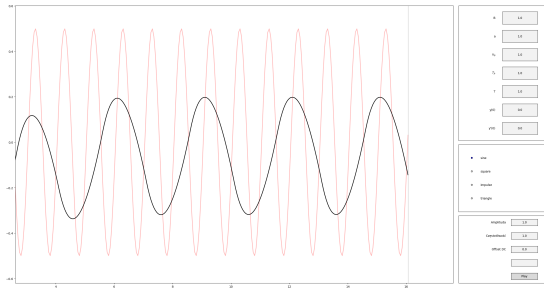
Kluczowym elementem algorytmu jest numeryczna realizacja członu nieliniowego w postaci przełącznika z histerezą o progach przełączania $\pm b$ i wartościach $\pm B$. Aby poprawnie odwzorować wieloznaczność charakterystyki histerezy, funkcja przełączająca zapamiętuje swój poprzedni stan i analizuje kierunek zmian sygnału uchybu, co zapobiega powstawaniu oscylacji numerycznych wewnątrz strefy nieczułości, do celu wyliczania współczynników k_i jest też możliwość manualnego podania stanu uznanego za poprzedni do funkcji symulującej wyjście histerezy.

Wymuszenia testowe podawane na wejście układu są generowane z wykorzystaniem biblioteki NumPy, która pozwala na wydajne, analityczne wyznaczanie wartości sygnałów dla dowolnej chwili czasowej t . Program oferuje pełną obsługę fali harmonicznego (sinusoidalnej), fali trójkątnej, fali prostokątnej z regulowanym współczynnikiem wypełnienia oraz pojedynczego impulsu o zdefiniowanym czasie rozpoczęcia i czasie trwania. Biblioteka NumPy także została wykorzystana do przechowywania historii sygnałów oraz stanu układu, co umożliwia szybki dostęp do danych i ich efektywne przetwarzanie podczas symulacji.

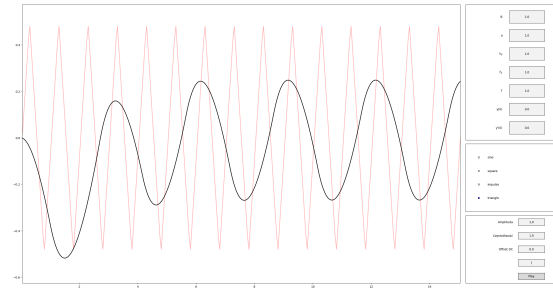
Warstwa wizualna i interaktywna programu została zbudowana na bazie biblioteki Matplotlib. Okno aplikacji zostało ustrukturyzowane przy użyciu funkcji `subplot_mosaic`, co pozwoliło na jednoczesne wyświetlanie głównego wykresu przebiegów czasowych oraz paneli sterowania. Płynność prezentacji danych w trybie online zapewnia mechanizm `FuncAnimation`, który odświeża wykres z częstotliwością 60 Hz. Podczas działania animacji okno obserwacji wykresu jest automatycznie przesuwane i utrzymuje stałą szerokość 15 jednostek czasu, podążając za najnowszymi punktami obliczonymi przez zaimplementowany silnik RK45. Użytkownik ma możliwość modyfikacji symulacji za pomocą wbudowanych widżetów: menu `RadioButtons` służy do zmiany typu sygnału wejściowego, natomiast zestaw pól `TextBox` pozwala na modyfikację warunków początkowych układu, parametrów liniowych (k_p, T_p, T) oraz nieliniowych (b, B) bez konieczności restartowania aplikacji. Modyfikowalne są także parametry sygnału wejściowego. Zmiana obiektu bądź sygnału wejściowego resetuje historię oraz rozpoczyna symulację od czasu $t = 0$. Dodatkowo zaimplementowano przycisk pauzy, umożliwiający wstrzymanie kroku animacji i dokładną analizę utrwalonego przebiegu.

4 Wyniki symulacji

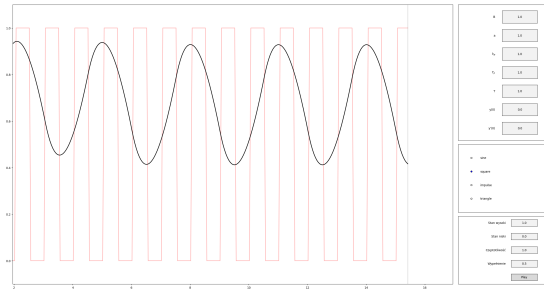
Poniżej przedstawiono przykładowe wyniki symulacji dla różnych sygnałów wejściowych oraz parametrów układu. Na wykresach można zaobserwować charakterystyczne zachowania układu nieliniowego, takie jak oscylacje, przeregulowania oraz stabilizację w zależności od rodzaju wymuszenia i ustawień parametrów.



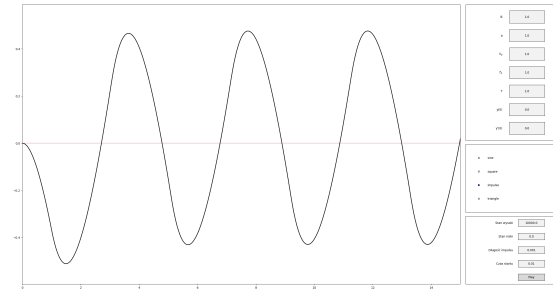
Rys. 3 Odpowiedź układu na sygnał sinusoidalny



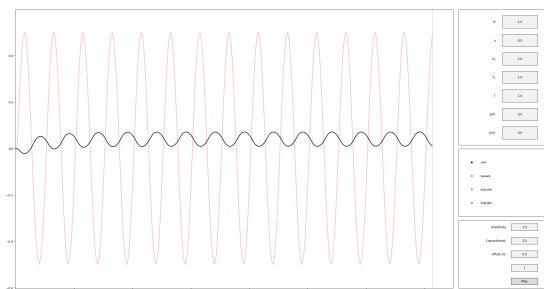
Rys. 4 Odpowiedź układu na sygnał trójkątny



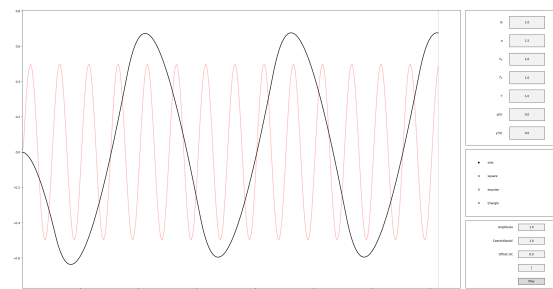
Rys. 5 Odpowiedź układu na sygnał prostokątny



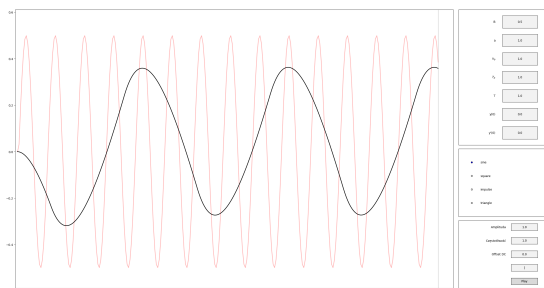
Rys. 6 Odpowiedź układu na pojedynczy impuls



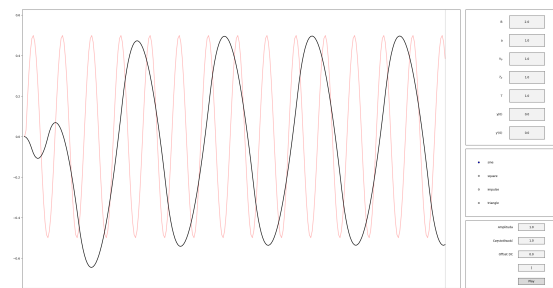
Rys. 7 Wpływ szerokości histerezy na wyjście układu ($b=0.5$)



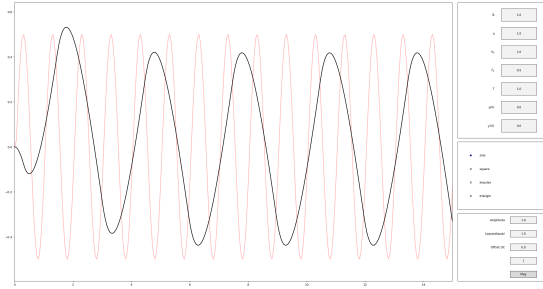
Rys. 8 Wpływ szerokości histerezy na wyjście układu ($b=1.5$)



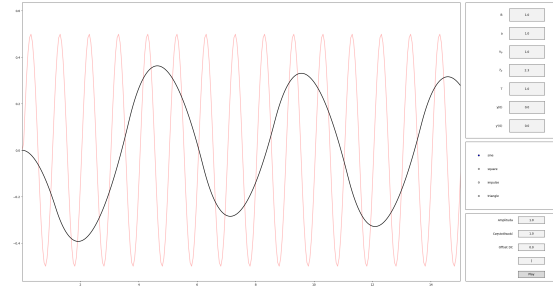
Rys. 9 Wpływ amplitudy histerezy na wyjście układu ($B=0.5$)



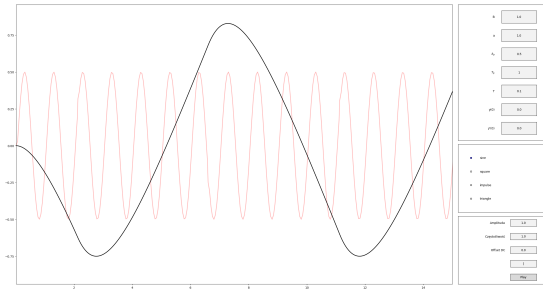
Rys. 10 Wpływ amplitudy histerezy na wyjście układu ($b=2.0$)



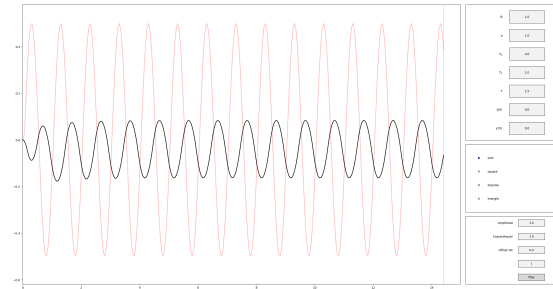
Rys. 11 Wpływ zmiany parametru T_p ($T_p = 0.5$)



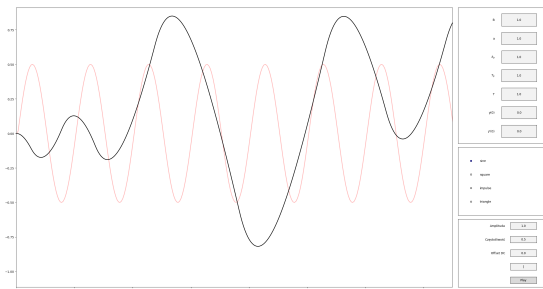
Rys. 12 Wpływ zmiany parametru T_p ($T_p = 2.3$)



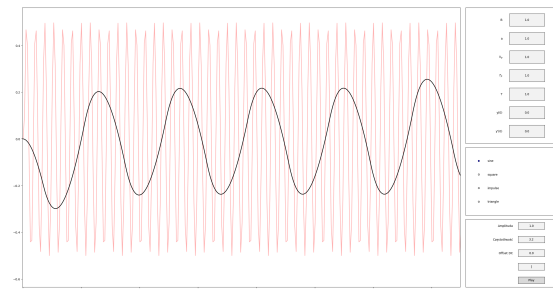
Rys. 13 Wpływ zmiany wzmocnień T oraz k_p (niskie wartości)



Rys. 14 Wpływ zmiany parametru T oraz k_p (wysokie wartości)



Rys. 15 Wpływ częstotliwości sygnału wejściowego (niskie częstotliwości)



Rys. 16 Wpływ częstotliwości sygnału wejściowego (wysokie częstotliwości)

5 Wnioski

Zrealizowany projekt umożliwia kompleksową analizę układów dynamicznych pod wpływem różnych wymuszeń. Zastosowanie metody Rungego-Kutty 4(5) pozwala na uzyskanie dokładnych wyników przy efektywnym czasie obliczeń.

Integracja GUI z biblioteką Matplotlib znacząco zwiększa funkcjonalność programu, umożliwiając intuicyjną i interaktywną analizę parametrów zarówno sygnału, jak i modelu układu.

Projekt spełnia założone cele i stanowi funkcjonalne narzędzie do symulacji danego układu dynamicznego. Dzięki wykorzystaniu Python oraz bibliotek NumPy i Matplotlib możliwe było stworzenie elastycznego i przejrzystego środowiska do analizy sygnałów i odpowiedzi układu.